TECHNICAL DESIGN DOCUMENT — V0.2

# Relay

A role-aware context layer that keeps your team's AI coding assistants in sync automatically — without anyone typing a status update.

---

**Track** — Developer Tools **Event** — iQOO Hackathon 2026, Hyderabad City Battle **Status** — Pre-build draft

CONTENTS


Problem statement

Core concept

System architecture

End-to-end sequence

Context relevance filtering

Voice command pipeline

Data model

Office Kit integration

Tech stack

MVP scope

Known risks

Scoring rubric fit


## 01 Problem statement

Every dev team using AI coding assistants has the same silent tax: each person's assistant only knows what that person told it. Nothing is shared between teammates unless a human explicitly explains it — in Slack, in standup, in a hallway conversation.

That re-explaining is pure overhead. A Coder fixes a function; the Debugger doesn't find out until something breaks and someone asks "did anything change here?" A Reviewer approves a PR without knowing the reasoning behind a tradeoff, because that reasoning lived in someone's head, not in the diff.

Relay doesn't replace Copilot, Cursor, or any assistant your team already uses. It sits underneath them — a thin context layer that keeps every teammate's assistant silently aware of what the others just did.

## 02 Core concept

Each teammate opens Relay on their phone and picks a role for the session: **Coder**, **Debugger**, or **Deployer**. Relay watches code activity (via an IDE extension, or GitHub as a fallback), decides what's actually worth surfacing to each other role, and pushes a filtered digest — silently, with no message sent by any human.

Voice is the primary interface. A teammate speaks a role-scoped command — `"Debugger, check the latest bug"` — which Relay parses on-device and forwards, along with the synced context, to whatever AI assistant that teammate already has connected.

## 03 System architecture

Four layers: capture (phone + IDE), the context engine (filtering + shared state), the bridge (Office Kit), and the assistant layer Relay routes into.

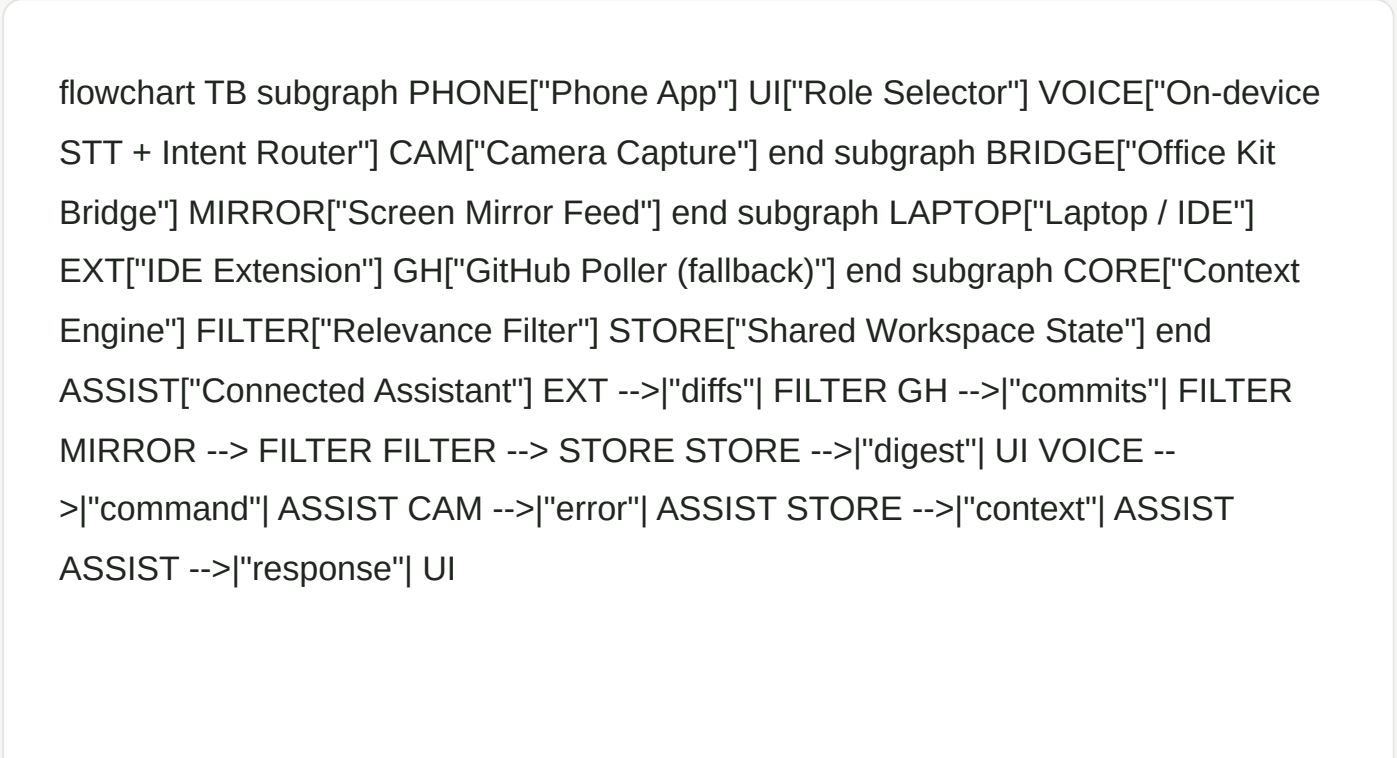


Fig 1 — Relay does not generate code. It captures, filters, and routes context between the team's existing assistants.

## 04 End-to-end sequence

The flagship demo moment: a change made by one teammate reaches another's assistant with zero manual communication.

sequenceDiagram participant C as Coder participant IDE as IDE Extension participant FE as Relevance Filter participant D as Debugger participant AI as Assistant C->>IDE: Save file / commit IDE->>FE: Diff + metadata FE->>FE: Classify, score FE-->>D: Push digest (silent) D->>D: "Debugger, check latest bug" D->>AI: Command + context AI-->>D: Diagnosis + fix D-->>FE: Push resolution FE-->>C: Silent notify — resolved

Fig 2 — Notice the Coder is never messaged directly. The digest reaches the Debugger's context before it's asked for.

## 05 Context relevance filtering

This is the actual hard problem in Relay. Syncing every diff verbatim is just noise — worse than Slack. The filter has to decide what's worth surfacing, and to whom.

flowchart TD A["Incoming diff"] --> B{"Change type?"} B -->|"Signature change"| C["High significance"] B -->|"Logic, no tests"| D["Medium significance"] B -->|"Formatting only"| E["Suppress"] C --> F["Generate digest"] D --> F E --> G["Store, no push"] F --> H{"Relevant to?"} H -->|"Error-prone area"| I["Debugger"] H -->|"Deploy readiness"| J["Deployer"] H -->|"Shared module"| K["Other Coders"]

Fig 3 — Significance scoring + role-relevance mapping, run by an on-device model before anything gets synced.

### Signal heuristics (v0)

| Signal | Weight | Rationale |
|---|---|---|
| Function signature changed | **High** | Breaks callers — always worth surfacing to anyone touching the same module |
| Error handling added/removed | **High** | Directly relevant to Debugger role by definition |
| New dependency introduced | **Medium** | Deploy-relevant; affects Deployer's readiness check |
| Logic change, tests untouched | **Medium** | Flag as "needs verification," not a hard alert |
| Rename, formatting, comments | **Suppress** | Zero cross-role value — this is what makes Relay not a raw commit feed |

## 06 Voice command pipeline

Command capture must work fully offline — both for reliability at a crowded venue, and because it's what actually counts as on-device AI for scoring.

| Stage | Runs on | Detail |
|---|---|---|
| Wake pattern | Phone, offline | Role name as trigger — `"Debugger, ..."` — avoids full always-on listening |
| Speech-to-text | Phone, offline | On-device Whisper Tiny or native Android STT |
| Intent parse | On-device LLM | Maps utterance to {role, action, target} |

| | | |
|---|---|---|
| Context attach | Context engine | Pulls latest relevant digest for that role |
| Route | Network call | Forwarded to the connected assistant with command + context |
| No assistant connected | Phone | Fails clearly: "You're not connected to any coding assistant." |

## 07 Data model

Minimal shared state — three entities carry the whole system.

| Entity | Key fields | Purpose |
|---|---|---|
| `WorkspaceEvent` | id, source, raw_diff, file, timestamp | Raw capture, pre-filter |
| `RelevanceDigest` | event_id, significance, summary, target_roles | Filtered, role-tagged output |
| `RoleSession` | member_id, role, connected_assistant | Tracks who's playing which role |

## 08 Office Kit integration

Office Kit is used for one concrete purpose, not as a vague "bridge": it mirrors the active IDE screen to the phone. That mirrored feed is what lets the phone-side context engine detect a save/change event and pull the relevant diff — without Office Kit, the phone has no visibility into what just happened on the laptop.

**WHY THIS MATTERS FOR SCORING**

Office Kit usage is measured via device data, not the pitch — so this has to be a real, active dependency during the build, not a claim. Screen-mirror-triggers-context-pull is the mechanism that makes that true.

## 09 Tech stack

| Layer | Choice |
|---|---|
| Phone app | Native Android (Kotlin) — direct Snapdragon NPU + Office Kit access |
| On-device STT | Whisper Tiny (quantized) or Android SpeechRecognizer, offline |
| On-device intent model | Small local LLM (Phi-3 Mini / Gemma 2B class), NPU-accelerated |
| IDE extension | VS Code extension — file-save + diff hooks |
| Fallback tracking | GitHub REST API polling |
| Context engine | Lightweight backend — event ingestion, filtering, state store |
| Assistant routing | Copilot extension API (single integration for the demo) |
| Optional voice reply | Cloud TTS (ElevenLabs / Sarvam) — non-critical polish |

## 10 MVP scope — build vs. roadmap

Everything below the line is real, spoken about, and deliberately not built live.

- Coder + Debugger roles, fully working end to end — BUILD
- Silent context sync between the two roles — BUILD
- On-device voice command + offline STT — BUILD
- Camera-based error capture (Debugger) — BUILD
- Office Kit screen-mirror trigger — BUILD
- Single assistant integration (Copilot) — BUILD
- Deployer role — ROADMAP

- GitHub fallback tracking — ROADMAP
- Cloud voice replies (ElevenLabs / Sarvam) — ROADMAP
- Multi-assistant support (Cursor, Continue, etc.) — ROADMAP

## 11 Known risks

HIGH

### Technical depth is thin without the filter

If filtering is naive, Relay is "call Copilot with extra steps." The significance scoring in §5 is the actual defensible engineering.

HIGH

### Live demo depends on external API

Copilot's response quality and venue wifi are outside Relay's control during the pitch window.

MEDIUM

### The best feature is invisible

Auto-sync is silent by design — the demo needs an explicit beat ("notice I said nothing") or judges miss it entirely.

MEDIUM

### Narrow adoption story

Strongest for remote/async teams; weaker for co-located teams sitting at the same table — name this explicitly in the pitch.

## 12 Scoring rubric fit

| Criterion | Weight | Relay's coverage |
|---|---|---|
| End product quality | 30% | Scoped to 2 roles only — one reliable loop, not five shaky ones |
| Novelty and impact | 20% | Silent role-to-role context propagation — not a chatbot, not a status bot |
| Creative phone use | 15% | Voice is the primary interface; camera captures errors directly |
| Technical depth | 15% | Relevance filtering + on-device intent routing (§5, §6) |

| | | |
|---|---|---|
| Office Kit usage | 10% | Screen-mirror is structurally required to detect changes — real usage, not decoration |
| Demo and presentation | 10% | Two phones, two people, one explicit "I said nothing" beat |